Chapter IV

Exercise 00 : ft_strlen

	Exercise 00
	ft_strlen
Turn-in directory: *ex*00/	
Files to turn in: `ft_strlen.c`	
Allowed functions: `None`	

- Create a function that counts and returns the number of characters in a string.
- The function should be prototyped as follows:

```
int    ft_strlen(char *str);
```

Chapter V

Exercise 01 : ft_putstr

	Exercise 01
ft_putstr	
Turn-in directory: $ex01/$	
Files to turn in: `ft_putstr.c`	
Allowed functions: `write`	

- Create a function that prints a string of characters to the standard output.
- The function should be prototyped as follows:

```
void    ft_putstr(char *str);
```

Chapter VI

Exercise 02 : ft_putnbr

	Exercise 02
ft_putnbr	
Turn-in directory: $ex02/$	
Files to turn in: `ft_putnbr.c`	
Allowed functions: `write`	

- Create a function that displays the number passed as a parameter. The function must be able to handle all possible values of the **int** type.
- The function should be prototyped as follows:

```
void ft_putnbr(int nb);
```

- Example usage:
  - `ft_putnbr(42)` outputs "42".

Chapter VII

Exercise 03 : ft_atoi

	Exercise 03
	ft_atoi
Turn-in directory: *ex*03/	
Files to turn in: `ft_atoi.c`	
Allowed functions: `None`	

- Write a function that converts the initial portion of the string pointed to by `str` into its integer representation.
- The string may begin with an arbitrary amount of whitespace (as determined by `isspace(3)`).
- The string may be preceded by an arbitrary number of `'+'` and `'-'` signs. A `'-'` sign will invert the result depending on whether the number of `'-'` signs is odd or even.
- The function should then process any consecutive digits in base 10.
- The function reads the string until a non-conforming character is encountered and returns the number obtained so far.
- Overflow and underflow do not need to be handled; the function's return value is undefined in such cases.
- **Example Program Output**:

```
$>./a.out "   ---+--+1234ab567"
-1234
```

- The function should be prototyped as follows:

```
int     ft_atoi(char *str);
```

Chapter VIII

Exercise 04 : ft_putnbr_base

	Exercise 04
ft_putnbr_base	
Turn-in directory: *ex*04/	
Files to turn in: `ft_putnbr_base.c`	
Allowed functions: `write`	

- Create a function that displays a number in a given base system in the terminal.
- The number is provided as an **int**, and the base is represented as a **string of characters**.
- The base system consists of all the symbols used to represent the number.
  - `0123456789` is the commonly used base system for representing decimal numbers.
  - `01` is a binary base system.
  - `0123456789ABCDEF` is a hexadecimal base system.
  - `poneyvif` is an octal base system.
- The function must handle negative numbers.
- If an invalid argument is provided, nothing should be displayed.
  Examples of invalid arguments:
  - The base is empty or has only one character.
  - The base contains duplicate characters.
  - The base contains `'+'` or `'-'`.

- The function should be prototyped as follows:

```
void        ft_putnbr_base(int nbr, char *base);
```

Chapter IX

Exercise 05 : ft_atoi_base

	Exercise 05
ft_atoi_base	
Turn-in directory: *ex*05/	
Files to turn in: `ft_atoi_base.c`	
Allowed functions: `None`	

- Write a function that converts the initial portion of the string pointed to by `str` into an integer representation.
- `str` is in a specific base, given as a second parameter.
- Except for the base rule, the function should behave exactly like **ft_atoi**.
- If an invalid argument is provided, the function should return 0.
  Examples of invalid arguments:
  - The base is empty or has only one character.
  - The base contains duplicate characters.
  - The base contains `'+'`, `'-'`, or whitespace characters.
- The function should be prototyped as follows:

```
int        ft_atoi_base(char *str, char *base);
```